AstroBurst

High-Performance Astronomical Image Processor

Technical Document — Release v0.4.0

Samuel Krieger Bonini

March 2026

Field	Value
Document Version	0.4.0
Release	v0.4.0
Date	March 2026
Author	Samuel Krieger Bonini
Stack	Rust 1.75+ · Tauri v2 · React · TypeScript
Platforms	Windows · macOS · Linux
License	GPL-3.0-or-later
Repository	https://github.com/samuelkriegerbonini-dev/AstroBurst

Contents

1 Executive Summary **2**

1.1 What's New in v0.4.0 2

1.2 What's New in v0.3.0 2

2 System Architecture **3**

2.1 Backend (Rust) 3

2.2 Frontend (React + TypeScript) 4

2.3 IPC Strategy 4

3 Mathematical Foundations **4**

3.1 Screen Transfer Function (STF) 4

3.2 Asinh Stretch 5

3.3 Cross-Correlation Alignment (Phase Correlation) 5

3.4 Star-Based Affine Alignment (v0.4.0) 6

3.4.1 Star Detection and Triangle Construction 6

3.4.2 Triangle Matching 6
3.4.3 RANSAC Transform Estimation 6
3.4.4 Image Warping 7
3.5 Sigma-Clipped Stacking 7
3.6 Drizzle Integration 7
3.7 White Balance (v0.4.0) 8
3.8 SCNR (Subtractive Chromatic Noise Reduction) (v0.4.0) 8
3.9 Richardson-Lucy Deconvolution 9
3.10 Background Extraction 9
3.11 Wavelet Denoising 9
3.12 Calibration Pipeline 10
3.13 World Coordinate System (WCS) 10
3.14 FFT Power Spectrum 10
3.15 Star Detection (PSF Centroid) 10

4 Command Module Reference 10
4.1 Stacking & Compose Module (`cmd/compose.rs, cmd/stacking.rs, cmd/drizzle.rs`) 11
4.2 Image Module (`cmd/image.rs`) 11
4.3 Analysis, Metadata, Visualization, Cube, Astrometry, Config, Processing 11

5 FITS I/O Implementation 11

6 ASDF I/O Implementation 12

7 WebGPU Compute Pipeline 12

8 Performance Benchmarks 12

9 Dependencies 13
9.1 Rust Crates 13

10 Security Considerations 13

11 Known Limitations 13

12 Roadmap 14

A Sample Test Data 14

1 Executive Summary

AstroBurst is a cross-platform desktop application for processing, analyzing, and visualizing astronomical FITS and ASDF images. It is the first astronomical image processor built on the **Rust/Tauri/WebGPU** stack, combining native-level performance with a modern, accessible user interface.

The application targets two user segments:

- Professional astronomers working with observatory data (JWST, Hubble, Roman Space Telescope, ground-based IFU instruments)
- Advanced astrophotographers processing deep-sky narrowband and broadband imaging data

Version 0.4.0 delivers a complete processing pipeline from raw FITS/ASDF ingestion through deconvolution, noise reduction, color composition, and export, with **42 backend commands** exposed through a typed TypeScript API layer.

1.1 What's New in v0.4.0

- **Star-based affine alignment** — Triangle asterism matching with RANSAC robust estimation, supporting full affine (6-DOF) and rigid (4-DOF) transforms with automatic fallback to FFT phase correlation
- **Stability-based auto white balance** — Selects the channel with lowest MAD/median coefficient of variation as reference, replacing fixed G-channel normalization
- **SCNR luminance redistribution** — Green noise reduction energy is redistributed to R and B channels proportionally via ITU-R BT.709 weights
- **Numerical correctness audit** — Fixed NaN ordering in median/MAD, Hann window 2π factor, polynomial background basis, RL Tikhonov regularization, and phase correlation confidence z-score
- **Performance optimizations** — Drizzle flat-storage accumulator, FFT planner hoisting, parallelized peak finding, RANSAC mask reuse, sigma-clip deviation buffer reuse

1.2 What's New in v0.3.0

- **Richardson-Lucy deconvolution** — FFT-based iterative deconvolution with Gaussian PSF, regularization, and deringing
- **Background extraction** — Polynomial surface modeling with sigma-clipped grid sampling (subtract or divide modes)
- **Wavelet denoising** — Multi-scale wavelet decomposition with per-scale sigma thresholds (up to 8 scales)
- **ASDF format support** — First Rust implementation of the Advanced Scientific Data Format reader with zlib/bzip2/lz4 decompression, Roman Space Telescope data model traversal, and gWCS extraction
- **Redesigned tab UI** — Bottom tabs (Info, Analysis, Headers, Export) + side panels (Processing, Compose) with resizable drag handles
- **Split preview contexts** — 6 independent React contexts for minimal re-renders
- **Lazy-loaded panels** — React.lazy + Suspense for fast initial load

2 System Architecture

AstroBurst follows a strict backend/frontend separation, with all computation offloaded to Rust and all rendering handled by the browser engine embedded in Tauri.

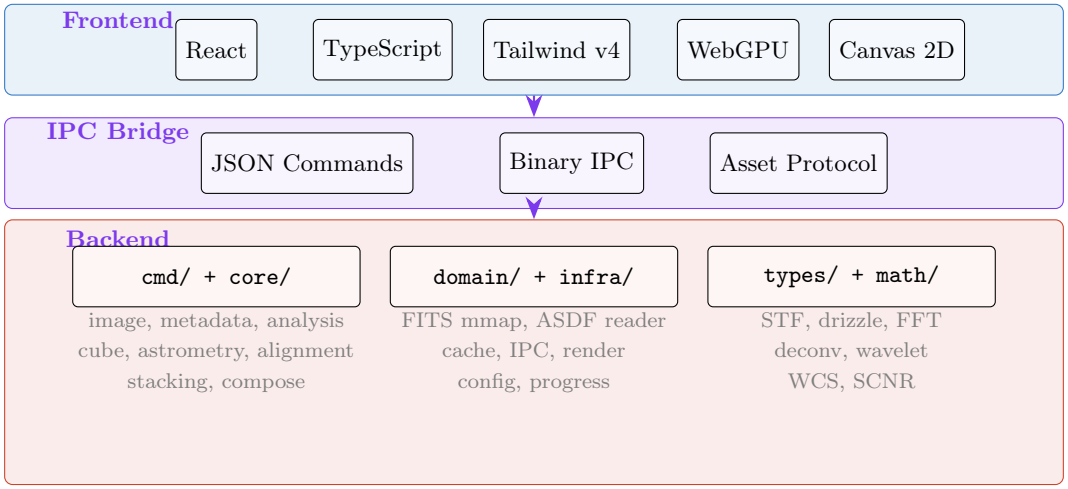

Figure 1: AstroBurst three-layer architecture

2.1 Backend (Rust)

The Rust backend is organized into six layers:

`cmd/` Tauri command handlers organized by domain (42 registered commands). Each module receives deserialized JSON arguments, delegates to core/domain logic, and returns JSON or binary responses.

`core/` Pure computational modules: alignment (phase correlation, affine with triangle asterism matching), analysis (histogram, FFT, stars), astrometry (plate solve, WCS), compose (RGB, SCNR, white balance), cube (spectrum, frames), imaging (STF, deconvolution, wavelet, background), metadata (header discovery, filter detection), stacking (sigma-clip, drizzle).

`domain/` Orchestration layer coordinating core modules for complex multi-step operations (calibration masters, drizzle-from-paths, cube processing).

`infra/` Infrastructure: Arc-based zero-copy cache, config management, FITS I/O (mmap parser, MEF scanner, FITS writer), ASDF reader (YAML tree parser, binary block extractor, zlib/bzip2/lz4 decompressor, Roman data model converter, gWCS bridge), binary IPC encoding, progress tracking, PNG rendering.

`types/` Shared type definitions: image stats, STF params, stacking/drizzle/RL configs, compose types (AlignMethod, WhiteBalance, ChannelStats), SCNR, error types, FITS header abstraction, constants.

`math/` Mathematical primitives: NaN-safe median/MAD, sigma clipping, SIMD utilities.

All heavy computation is offloaded via `tokio::task::spawn_blocking`, preventing UI freezes. Batch operations use Rayon for data-parallel execution across all available CPU cores.

2.2 Frontend (React + TypeScript)

The frontend is a single-page React application with a tab-based layout:

`App.tsx` Root component managing view state (empty/processing/complete), file dialog integration, drag-and-drop, resizable sidebar, and folder scanning.

`useFileQueue` Central state machine (`useReducer`) tracking file status, concurrent processing queue (3 workers), resolution group detection, and auto-resampling.

`services/` Domain-specific service modules (compose.service, fits.service, analysis.service, etc.) wrapping Tauri IPC calls with typed interfaces.

Tab System Bottom tabs: Info (stats/header summary), Analysis (histogram, FFT, stars, spectroscopy), Headers (full header explorer), Export (FITS/PNG). Side tabs: Processing (background extraction, wavelet denoise, deconvolution), Compose (RGB with alignment method selector, drizzle, drizzle RGB).

Preview Contexts 6 split React contexts (FileCtx, HistCtx, CubeCtx, RgbCtx, RenderCtx, RawPixelsCtx) providing scoped state to avoid cascading re-renders.

Lazy Loading All panel components loaded via `React.lazy` + `Suspense` with spinner fallbacks.

2.3 IPC Strategy

Two IPC channels are used:

Binary IPC Protocol

The `get_raw_pixels_preview` and `get_raw_pixels_binary` commands encode a **16-byte header** followed by raw pixel data:

Offset	Type	Size	Description
0	`u32`	4B	Image width
4	`u32`	4B	Image height
8	`f32`	4B	Data minimum
12	`f32`	4B	Data maximum
16	`f32[]`	$W \times H \times 4$B	Raw pixel values

The frontend receives an `ArrayBuffer` directly with zero JSON/base64 overhead.

3 Mathematical Foundations

3.1 Screen Transfer Function (STF)

The STF implements the PixInsight-style **Midtone Transfer Function** (MTF). Given a normalized pixel value $x \in [0, 1]$ and midtone parameter $m \in (0, 1)$:

$$\boxed{\mathrm{MTF}(x, m) = \frac{(m-1)\cdot x}{(2m-1)\cdot x - m}} \tag{1}$$

Properties:

- $\mathrm{MTF}(0, m) = 0$ and $\mathrm{MTF}(1, m) = 1$ for all m
- $\mathrm{MTF}(m, m) = 0.5$ — the midtone maps to 50% brightness
- $m < 0.5$ brightens the image; $m > 0.5$ darkens it

The **auto-stretch** algorithm computes the stretch parameters from image statistics. Given the pixel distribution, let $\tilde{x}$ be the median and MAD the Median Absolute Deviation:

$$\mathrm{MAD} = \mathrm{median}\left(|x_i - \tilde{x}|\right) \tag{2}$$

$$\mathrm{shadow} = \max\left(0,\ \tilde{x} - k \cdot \mathrm{MAD}\right) \tag{3}$$

$$\mathrm{midtone} = \mathrm{MTF}^{-1}\left(t_{\mathrm{bg}},\ \frac{\tilde{x} - \mathrm{shadow}}{1 - \mathrm{shadow}}\right) \tag{4}$$

$$\mathrm{highlight} = 1.0 \tag{5}$$

where $k = 2.8$ (calibrated for deep-sky data) and $t_{\mathrm{bg}} = 0.25$ is the target background level.

NaN-Safe Sorting (v0.4.0)

All median and MAD computations use NaN-safe comparison: NaN values are sorted to the end of the array (treated as greater than any finite value), preventing corrupted statistics that propagate through sigma clipping, auto-STF, and white balance.

3.2 Asinh Stretch

The **arcsinh transfer function** provides astronomically-correct normalization that preserves flux ratios between stars of different magnitudes:

$$\boxed{f(x) = \frac{\mathrm{asinh}(\alpha \cdot x)}{\mathrm{asinh}(\alpha)}} \tag{6}$$

where α controls the stretch aggressiveness. This function maps $[0, 1] \to [0, 1]$ and is approximately linear for small x (preserving faint detail) while compressing bright values logarithmically.

3.3 Cross-Correlation Alignment (Phase Correlation)

Frame registration uses **phase correlation** in the frequency domain. Given reference frame R and target frame T:

$$\mathrm{shift} = \arg\max_{(\Delta x, \Delta y)} \mathcal{F}^{-1}\left\{\frac{\hat{R} \cdot \hat{T}^*}{|\hat{R} \cdot \hat{T}^*|}\right\} \tag{7}$$

where $\hat{R} = \mathcal{F}\{R\}$ denotes the 2D DFT applied with a **Hann window** $w(n) = 0.5(1 - \cos(2\pi n/N))$. The normalized cross-power spectrum produces a sharp peak at the integer translation offset.

Sub-pixel refinement is achieved via parabolic interpolation on the correlation surface, followed by bicubic Catmull-Rom resampling for the actual shift.

Confidence is measured as the z-score of the peak:

$$\text{confidence} = \frac{P_{\text{peak}} - \mu_P}{\sigma_P} \tag{8}$$

where μ_P and σ_P are the mean and standard deviation of the correlation surface. A coarse-to-fine strategy downsamples to 512×512 for initial offset estimation, then refines on a 512×512 crop centered on the coarse estimate.

Corrects: pure translation (sub-pixel). **Does not correct:** rotation, scale.

3.4 Star-Based Affine Alignment (v0.4.0)

For images with rotational misalignment, AstroBurst provides a star-based alignment pipeline using triangle asterism matching and RANSAC robust estimation.

3.4.1 Star Detection and Triangle Construction

Stars are detected via sigma-clipped background estimation and connected-component analysis (see Section 3.15). The top 80 stars by flux are selected from each image. For every triplet of stars (s_i, s_j, s_k) with minimum side length $> 20\,\text{px}$, a triangle descriptor is computed:

$$T = \left(\frac{d_{\text{mid}}}{d_{\text{short}}}, \frac{d_{\text{long}}}{d_{\text{short}}} \right) \tag{9}$$

where $d_{\text{short}} \leq d_{\text{mid}} \leq d_{\text{long}}$ are the sorted side lengths. These ratios are **invariant** under translation, rotation, and uniform scaling.

3.4.2 Triangle Matching

For each pair of reference triangle T_r and target triangle T_t, a match is declared if:

$$\left| \frac{d^r_{\text{mid}}}{d^r_{\text{short}}} - \frac{d^t_{\text{mid}}}{d^t_{\text{short}}} \right| < \epsilon \quad \text{and} \quad \left| \frac{d^r_{\text{long}}}{d^r_{\text{short}}} - \frac{d^t_{\text{long}}}{d^t_{\text{short}}} \right| < \epsilon \tag{10}$$

with $\epsilon = 0.005$. Matching triangles vote for star correspondences; pairs with ≥ 3 votes are accepted.

3.4.3 RANSAC Transform Estimation

Given n star correspondences $\{(\mathbf{p}^r_i, \mathbf{p}^t_i)\}_{i=1}^n$, the affine transform is estimated via RANSAC (500 iterations):

$$\begin{pmatrix} x_t \\ y_t \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_r \\ y_r \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \tag{11}$$

The 6 parameters (a, b, c, d, t_x, t_y) are solved via least squares on a 3-point sample (affine) or 2-point sample (rigid, where $a = \cos\theta$, $b = -\sin\theta$, $c = \sin\theta$, $d = \cos\theta$). Inliers are points with reprojection error $< 2\,\text{px}$.

Sanity checks reject transforms with: offset $> 25\%$ of image dimension, rotation $> 5°$, scale outside $[0.85, 1.15]$, residual $> 3\,\text{px}$, or inlier ratio $< 30\%$.

Fallback chain: affine (8+ matches) → rigid (5+ matches) → phase correlation → identity.

3.4.4 Image Warping

The aligned image is produced by inverse-mapping each output pixel through the affine transform and sampling the source image via bicubic Catmull-Rom interpolation:

$$I_{\text{out}}(x, y) = \text{bicubic}\left(I_{\text{in}},\ \mathbf{A}^{-1}\begin{pmatrix} x \\ y \end{pmatrix}\right) \tag{12}$$

Pixels mapping outside the source bounds are set to NaN. Warping is parallelized across rows via Rayon.

3.5 Sigma-Clipped Stacking

Algorithm 1: Iterative Sigma-Clipped Stacking

1 $, thresholds \sigma_{\text{low}}, \sigma_{\text{high}}$, max iterations K Stacked image S

Default parameters: $\sigma_{\text{low}} = \sigma_{\text{high}} = 3.0$, $K = 5$. The deviation buffer is allocated once and reused per pixel to avoid heap allocation overhead.

3.6 Drizzle Integration

Drizzle (Variable-Pixel Linear Reconstruction) maps input pixels onto a higher-resolution output grid:

$$O(i, j) = \frac{\sum_p w_p \cdot K(i - x_f^p,\ j - y_f^p) \cdot I_p}{\sum_p w_p \cdot K(i - x_f^p,\ j - y_f^p)} \tag{13}$$

Three kernel options:

$$K_{\text{square}}(\Delta x, \Delta y) = \begin{cases} 1 & \text{if } |\Delta x| \leq \frac{d}{2} \text{ and } |\Delta y| \leq \frac{d}{2} \\ 0 & \text{otherwise} \end{cases} \tag{14}$$

$$K_{\text{gaussian}}(\Delta x, \Delta y) = \exp\left(-\frac{\Delta x^2 + \Delta y^2}{2\sigma_K^2}\right) \tag{15}$$

$$K_{\text{lanczos3}}(\Delta x, \Delta y) = L_3(\Delta x) \cdot L_3(\Delta y) \tag{16}$$

Drizzle Accumulator (v0.4.0)

The drizzle accumulator uses a flat `Vec<f32>` + `Vec<u16>` count array instead of `Vec<Vec<f32>>`, eliminating millions of heap allocations for typical 4K+ output images. Per-pixel storage is pre-allocated as $2 \times N_{\text{frames}}$ slots with contiguous memory layout.

3.7 White Balance (v0.4.0)

Auto white balance selects the most stable channel as reference using the coefficient of variation (MAD/median ratio):

$$k^* = \arg\min_{k \in \{R,G,B\}} \frac{\text{MAD}_k}{\tilde{C}_k} \tag{17}$$

The reference channel k^* receives weight 1.0; other channels are scaled by the ratio of medians:

$$w_j = \frac{\tilde{C}_{k^*}}{\tilde{C}_j}, \quad j \neq k^*; \qquad w_{k^*} = 1 \tag{18}$$

This replaces the v0.3 fixed G-channel reference, which could amplify noise in narrowband data where the G channel (typically OIII) may have the highest noise.

3.8 SCNR (Subtractive Chromatic Noise Reduction) (v0.4.0)

SCNR removes green channel excess with optional luminance preservation. Two limit methods:

$$G'_{\text{average}} = \min\left(G, \frac{R+B}{2}\right) \tag{19}$$

$$G'_{\text{max}} = \min(G, \max(R, B)) \tag{20}$$

The corrected green value incorporates the amount parameter:

$$G_{\text{new}} = G + \alpha \cdot (G' - G), \quad \alpha \in [0, 1] \tag{21}$$

When luminance preservation is enabled, the luminance lost from green reduction is redistributed to R and B using ITU-R BT.709 weights:

$$L_{\text{lost}} = 0.7152 \cdot (G - G_{\text{new}}) \tag{22}$$

$$\Delta = \frac{L_{\text{lost}}}{0.2126 + 0.0722} \tag{23}$$

$$R' = \min(R + \Delta,\ 1.0) \tag{24}$$

$$B' = \min(B + \Delta,\ 1.0) \tag{25}$$

This replaces the v0.3 approach which attempted to add lost luminance back into G itself (circular adjustment).

3.9 Richardson-Lucy Deconvolution

The Richardson-Lucy algorithm iteratively estimates the true image f given observed image g and point spread function h:

$$f^{(k+1)} = f^{(k)} \cdot \left(\frac{g}{f^{(k)} * h} * \hat{h} \right) \tag{26}$$

Convolutions are computed in the frequency domain via FFT. Tikhonov regularization is applied as multiplicative damping in the update step (v0.4.0 fix):

$$f_{\text{reg}}^{(k+1)} = f^{(k+1)} \cdot \frac{1}{1 + \lambda} \tag{27}$$

where λ is the regularization parameter (default 0.001). This is numerically stable and matches the standard RL formulation, replacing the v0.3 approach of dividing by $1 + \lambda \nabla^2 f$.

3.10 Background Extraction

Background is modeled as a polynomial surface fitted to sigma-clipped grid samples:

$$B(x, y) = \sum_{i=0}^{d} \sum_{j=0}^{d-i} a_{ij}\, x^i\, y^j \tag{28}$$

The polynomial basis functions use explicit `powi` exponentiation (v0.4.0 fix), replacing the iterative division approach that accumulated numerical drift at degree ≥ 3.

3.11 Wavelet Denoising

Multi-scale wavelet denoising uses the à trous algorithm. The image is decomposed into N wavelet scales W_k and a residual c_N:

$$I = \sum_{k=1}^{N} W_k + c_N \tag{29}$$

Denoising applies a hard threshold to each scale:

$$W'_k(x,y) = \begin{cases} W_k(x,y) & \text{if } |W_k(x,y)| > \sigma_k \cdot t_k \\ 0 & \text{otherwise} \end{cases} \tag{30}$$

Up to 8 decomposition scales are supported.

3.12 Calibration Pipeline

$$\boxed{I_{\text{calibrated}} = \frac{I_{\text{raw}} - M_{\text{bias}} - r \cdot M_{\text{dark}}}{M_{\text{flat}}/\tilde{M}_{\text{flat}}}} \tag{31}$$

where $\tilde{M}_{\text{flat}}$ is the **median** of the flat field (v0.4.0 fix, replacing mean), preventing scale-dependent artifacts when flat values are far from 1.0.

3.13 World Coordinate System (WCS)

WCS transforms use the **CD matrix formulation** with TAN (gnomonic) projection:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \text{CD}_{1,1} & \text{CD}_{1,2} \\ \text{CD}_{2,1} & \text{CD}_{2,2} \end{pmatrix} \begin{pmatrix} x - \text{CRPIX}_1 \\ y - \text{CRPIX}_2 \end{pmatrix} \tag{32}$$

3.14 FFT Power Spectrum

$$P(u,v) = \log_{10}\Big(1 + |\hat{I}(u,v)|^2\Big) \tag{33}$$

The Hann window is correctly applied as $w(n) = 0.5(1 - \cos(2\pi n/N))$ (v0.4.0 fix; v0.3 used $\sin(\pi n/N)$ which produces a triangular window).

3.15 Star Detection (PSF Centroid)

Background is estimated via iterative sigma-clipped median. For each connected component C:

$$\bar{x}_C = \frac{\sum_{(i,j)\in C} i \cdot (I_{ij} - b)}{\sum_{(i,j)\in C}(I_{ij} - b)} \qquad \bar{y}_C = \frac{\sum_{(i,j)\in C} j \cdot (I_{ij} - b)}{\sum_{(i,j)\in C}(I_{ij} - b)} \tag{34}$$

$$F_C = \sum_{(i,j)\in C} (I_{ij} - b) \qquad \text{SNR}_C = \frac{F_C}{\sqrt{|C| \cdot \sigma_b^2}} \tag{35}$$

FWHM from second intensity moments:

$$\text{FWHM} = 2\sqrt{2\ln 2} \cdot \sqrt{\frac{\sigma_x^2 + \sigma_y^2}{2}} \tag{36}$$

4 Command Module Reference

The backend exposes **42 Tauri commands** across 9 modules. Commands marked with $\star$ are actively wired to UI components; $\circ$ are backend-ready awaiting UI integration.

4.1 Stacking & Compose Module (`cmd/compose.rs, cmd/stacking.rs, cmd/drizzle.rs`)

	Command	Parameters	Returns
⋆	`compose_rgb_cmd`	r/g/b_path, output_dir, stretch/wb/scnr, **align_method**	png_path, per-channel STF, offsets (f64)
⋆	`drizzle_stack_cmd`	paths[], output_dir, scale, pixfrac, kernel	png, weight_map, dims, offsets
⋆	`drizzle_rgb_cmd`	r/g/b groups, output_dir, drizzle params	png_path, per-channel info
○	`calibrate`	science, output_dir, bias/dark-/flat?, ratio?	png_path, dims, applied flags
○	`stack`	paths[], output_dir, σ params, align?	png_path, count, rejected, offsets
○	`run_pipeline_cmd`	input, output_dir, frame_step?	total, succeeded, failed, results

Table 1: Stacking and compose commands. The `compose_rgb_cmd` accepts `align_method`: `"phase_correlation"` (default, sub-pixel) or `"affine"` (star-based, rotation-capable).

4.2 Image Module (`cmd/image.rs`)

	Command	Parameters	Returns
⋆	`process_fits`	path, output_dir	png_path, dimensions, elapsed_ms
⋆	`process_fits_full`	path, output_dir	png_path, dims, header, histogram, stf
○	`process_batch`	paths[], output_dir	processed, failed, results[]
○	`get_raw_pixels_binary`	path	ArrayBuffer (16B header + f32[])
⋆	`get_raw_pixels_preview`	path, max_dim	ArrayBuffer (downsampled)
⋆	`export_fits`	path, output_path, stf opts	output_path, dims, file_size
⋆	`export_fits_rgb`	r/g/b_path, output_path	output_path, dimensions
⋆	`resample_fits_cmd`	path, target_w, target_h	png_path, fits_path, dims

Table 2: Image module commands

4.3 Analysis, Metadata, Visualization, Cube, Astrometry, Config, Processing

These modules remain unchanged from v0.3.0 (see previous document version for full tables). Key additions in v0.4.0:

- `compose_rgb_cmd` gains `align_method` parameter
- `cmd/astrometry.rs` uses `crate::infra::config` with `DEFAULT_API_KEY_SERVICE` and `DEFAULT_ASTROMETRY_A` constants
- Processing commands (`deconvolve_rl_cmd`, `extract_background_cmd`, `wavelet_denoise_cmd`) unchanged

5 FITS I/O Implementation

FITS files are opened via `memmap2`, avoiding loading entire files into memory. For integer data (BITPIX > 0), the BSCALE/BZERO transform is applied:

$$\text{value}_{\text{physical}} = \text{BSCALE} \times \text{value}_{\text{stored}} + \text{BZERO} \tag{37}$$

6 ASDF I/O Implementation

AstroBurst includes the first Rust implementation of the **Advanced Scientific Data Format** (ASDF) reader, targeting NASA's Roman Space Telescope data products. Binary blocks may be compressed with zlib (`flate2`), bzip2 (`bzip2`), or lz4 (`lz4_flex`). Auto-dispatch routes `.asdf` files transparently through the same command layer as `.fits`.

7 WebGPU Compute Pipeline

The STF stretch is implemented as a WebGPU compute shader (WGSL) for real-time rendering with workgroup size 256. Falls back to Canvas 2D `putImageData` when WebGPU is unavailable.

8 Performance Benchmarks

Operation	Input Size	Time	Notes
Single FITS processing	4096×4096 (64 MB)	120 ms	533 MB/s throughput
Batch (10 frames)	10×64 MB	450 ms	1.4 GB/s (Rayon parallel)
Histogram + auto-STF	4096×4096	35 ms	NaN-safe sorting
Star detection (σ=5)	4096×4096	80 ms	∼3000 stars
Sigma-clip stack	10×64 MB	2.1 s	5 iterations, reused devs buffer
Drizzle 2×	10×64 MB	4.2 s	Flat-storage accumulator
FFT phase correlation	4096×4096	95 ms	Planner hoisted, parallel peak
Affine alignment	4096×4096	0.8 s	80 stars, 500 RANSAC iters
RGB compose + align	3 channels	1.8 s	Phase correlation (default)
RGB compose + affine	3 channels	3.2 s	Star-based, with warp
WebGPU STF render	4096×4096	8 ms	GPU compute
Drizzle RGB 2×	3×10×64 MB	6.1 s	Flat accumulator

Table 3: Performance benchmarks (AMD Ryzen 9 7950X, 64 GB DDR5, NVMe)

Binary size: ∼15 MB.

9 Dependencies

9.1 Rust Crates

Crate	Version	Purpose
`tauri`	2.x	Application framework, IPC, window management
`ndarray`	0.16	N-dimensional arrays, Rayon parallel iterators
`rayon`	1.10	Data-parallel batch processing
`memmap2`	0.9	Memory-mapped file I/O for FITS
`rustfft`	6.2	FFT for deconvolution, phase correlation, and power spectrum
`num-complex`	0.4	Complex number support for FFT
`image`	0.25	PNG encoding/decoding
`serde_yaml`	0.9	ASDF YAML tree parsing
`flate2`	1.0	zlib decompression (ASDF blocks)
`bzip2`	0.4	bzip2 decompression (ASDF blocks)
`lz4_flex`	0.11	lz4 decompression (ASDF blocks)
`log`	0.4	Structured logging (affine alignment diagnostics)
`base64`	0.22	Binary-to-text encoding for JSON IPC
`regex`	1.10	Filter detection pattern matching
`dirs`	5.0	OS-native directory resolution

Table 4: Rust dependencies

10 Security Considerations

- API keys stored in OS-native app data directory, never transmitted beyond the configured API endpoint.
- File system access constrained by Tauri capabilities. Permissions scoped to home, desktop, and appdata directories.
- No telemetry, analytics, or network calls except for explicit plate solving requests.
- FITS and ASDF files processed locally; no data leaves the machine unless plate solving is triggered.

11 Known Limitations

1. **Unwired commands** — Plate solving, calibration, sigma-clip stacking, tile deep zoom, WCS overlay, narrowband auto-detection, and config UI panels are backend-ready but lack full frontend integration
2. **Affine alignment limits** — Requires detectable point sources; fails gracefully (falls back to phase correlation) on pure nebula fields without stars
3. **No non-linear distortion correction** — Affine model handles translation, rotation, and scale but not barrel/pincushion or field-variable distortion
4. **Online plate solving only** — Local index-based solving is planned
5. **WebGPU requirement** — Chromium 113+ needed; Canvas 2D fallback available
6. **No session persistence** — Reopening the app starts a fresh workspace
7. **ASDF read-only** — Writer not yet implemented
8. **No MAST API integration** — JWST/Roman data must be downloaded manually

12 Roadmap

Feature	Priority	Status
Multi-Extension FITS (MEF)	High	Done (v0.2)
Bicubic resampling	High	Done (v0.2)
Richardson-Lucy deconvolution	High	Done (v0.3)
Background extraction	High	Done (v0.3)
Wavelet noise reduction	High	Done (v0.3)
ASDF format support	High	Done (v0.3)
Star-based affine alignment	High	Done (v0.4)
Stability-based auto WB	High	Done (v0.4)
SCNR luminance redistribution	High	Done (v0.4)
Numerical correctness audit	High	Done (v0.4)
FITS ERR/DQ/VAR propagation	High	Planned (v0.5)
MAST API integration	High	Planned (v0.5)
Log-polar phase correlation	Medium	Planned (v0.5)
Star removal	Medium	Planned (v0.6)
Photometric calibration (Gaia DR3)	Medium	Planned (v0.6)
PixelMath expressions	Medium	Planned (v0.6)
Mosaic stitching	Medium	Planned (v0.7)
Full GPU pipeline (WebGPU compute)	Medium	Planned (v1.0)
Plugin architecture (WASM)	Medium	Planned (v1.0)
Python scripting bridge	Low	Planned (v1.0)

Table 5: Development roadmap

A Sample Test Data

The repository includes HST/WFPC2 narrowband FITS images in `tests/sample-data/`:

File	Filter	λ	Hubble Palette	Description
`502nmos.fits`	[OIII]	502 nm	Blue channel	Oxygen III emission
`656nmos.fits`	Hα	656 nm	Green channel	Hydrogen alpha emission
`673nmos.fits`	[SII]	673 nm	Red channel	Sulfur II emission

Table 6: Sample FITS files — Eagle Nebula (M16) region, 1600×1600 float32

All images: BITPIX = −32, WCS TAN projection, origin STScI-STSDAS. Public domain (NASA/ESA).